

Die Jacobi-Matrix als universale Ersetzung des Gradienten: Eine formale Untersuchung der Äquivalenz und Verallgemeinerung

Stephan Epp

May 28, 2026

Abstract

Der Gradient ∇f ist eine zentrale Konzept der multivariaten Analysis mit ubiquitären Anwendungen in Optimierung, numerischen Methoden und maschinellem Lernen. Allerdings ist der Gradient nur für skalare Funktionen $f : \mathbb{R}^n \rightarrow \mathbb{R}$ definiert. Diese Arbeit zeigt formal und anschaulich, dass die **Jacobi-Matrix** eine vollständige, universale und strukturell äquivalente Ersetzung des Gradienten darstellt — und dies sogar für beliebige vektorwertige Funktionen $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$. Wir zeigen durch Theoreme und formale Beweise, wie jede Eigenschaft des Gradienten durch die Jacobi-Matrix wiedererlangt werden kann, und demonstrieren die Äquivalenz in Optimierung, Physik, Machine Learning und numerischer Analysis. Die Arbeit schließt mit einem ausführlichen Ausblick auf moderne Anwendungen, von automatischer Differenziation über topologische Datenanalyse bis zu symbolischen und algebraischen Methoden.

Contents

1	Einführung	2
1.1	Motivation und Problemstellung	2
1.2	Historischer Kontext	2
2	Grundlagen	2
2.1	Grundlegende Definitionen	2
3	Äquivalenz und Ersetzung: Der Gradient als Spezialfall	3
3.1	Theorem 1: Der Gradient ist die erste Zeile der Jacobi-Matrix	3
3.2	Theorem 2: Ersetzung des Gradienten in der Richtungsableitung	4
3.3	Theorem 3: Ersetzung in der Kettenregel	4
3.4	Theorem 4: Ersetzung in der Optimierung	4
4	Verallgemeinerung: Jacobi-Matrix für vektorwertige Funktionen	5
4.1	Theorem 5: Jacobi-Matrix als Erweiterung des Gradienten	5
4.2	Anwendung: Nicht-lineare Systeme	5

5	Numerische Beispiele und Vergleiche	5
5.1	Beispiel 1: Einfache quadratische Funktion	5
5.2	Beispiel 2: Vektorwertige Funktion	6
5.3	Beispiel 3: Newton-Raphson-Verfahren	6
6	Anwendungen: Jacobi-Matrix über verschiedene Disziplinen	7
6.1	1. Optimierung und Machine Learning	7
6.2	2. Physik: Vektorfelder und Divergenz	7
6.3	3. Numerische Analysis und Implizite Funktionen	7
6.4	4. Kontinuierliche Dynamische Systeme	7
7	Visualisierung und matplotlib-Plots	8
7.1	Plot 1: Quadratische Funktion mit Gradienten-Vektorfeld	8
7.2	Plot 2: 3D-Oberflächendarstellung	8
7.3	Plot 3: Äquivalenz von Gradient und Jacobi-Matrix	9
7.4	Plot 4: Vektorwertige Funktion	9
7.5	Plot 5: Gradientenabstieg — äquivalente Jacobi-Formulierung	10
7.6	Plot 6: Spektralanalyse der Hessian-Matrix	10
7.7	Plot 7: Impliziter Funktionssatz	11
7.8	Plot 8: Zusammenfassung — Äquivalenz und Verallgemeinerung	11
8	Theorem 6: Struktur-Erhaltung unter der Ersetzung	13
9	Theorem 7: Information-Äquivalenz	13
10	Kritische Diskussion: Wann ist der Gradient vorzuziehen?	14
10.1	Gradient-Vorteile	14
10.2	Jacobi-Matrix-Vorteile	14
11	Ausführlicher Ausblick: Zukunftsperspektiven	15
11.1	1. Automatische Differenziation (AD)	15
11.2	2. Differenzialgeometrie auf Mannigfaltigkeiten	15
11.3	3. Topologische Datenanalyse (TDA)	15
11.4	4. Algebraische Geometrie und Singularitätstheorie	15
11.5	5. Quantum Computing und Ableitungen	16
11.6	6. Hyperparameter-Optimierung und Zweite-Ableitungs-Information	16
11.7	7. Symbolische und Hybride Systeme	16
11.8	8. Robotik und Inverse Kinematics	16
11.9	9. Variationelle Methoden und funktionale Ableitungen	17
11.10	10. Open Research Questions	17
12	Anwendung in der Künstlichen Intelligenz: Optimale Jacobi-Matrix für Gewichtsmatrizen	18
12.1	Einleitung und Motivation	18
12.2	Formales Netzwerkmodell	18
12.3	Theorem 8: Minimale hinreichende Jacobi-Dimension	19
12.4	Theorem 9: Laufzeit der optimalen Gewichtsaktualisierung	20
12.5	Theorem 10: Rang-Schranke der adaptiven Jacobi-Matrix	22
12.6	Theorem 11: Vergleich mit naiver vollständiger Jacobi	23

12.7 Theorem 12: Adaptive Jacobi-Strategie für Adam-Optimierer	24
12.8 Zusammenfassung der Komplexitätsresultate	24
12.9 Numerisches Beispiel: GPT-2-ähnliches Netz	25
13 Zusammenfassung	26

1 Einführung

1.1 Motivation und Problemstellung

Der Gradient einer skalarwertigen Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$ ist definiert als:

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix} \in \mathbb{R}^n$$

Der Gradient ist eine fundamentale Struktur in:

- **Optimierung:** Gradientenabstieg minimiert Funktionen durch $x_{k+1} = x_k - \alpha \nabla f(x_k)$
- **Physik:** Kräfte sind negative Gradienten von Potentialen: $\vec{F} = -\nabla V$
- **Machine Learning:** Backpropagation basiert auf Gradientenberechnung
- **Numerik:** Newton-Verfahren verwendet Gradienten und Hessian-Matrizen

Zentrale Frage dieser Arbeit:

Kann die Jacobi-Matrix den Gradienten vollständig ersetzen? Welche Struktur geht verloren, welche bleibt erhalten? Und können wir diese Ersetzung auf vektorwertige Funktionen verallgemeinern?

1.2 Historischer Kontext

Die Jacobi-Matrix wurde 1841 von Carl Gustav Jacob Jacobi eingeführt und ist heute ein Standardkonzept der Multivariaten Analysis. Der Gradient als separates Konzept entstand später und wurde im 20. Jahrhundert zum primären Werkzeug der Optimierung und numerischen Analysis.

Lücke in der Literatur: Während Lehrbücher Gradient und Jacobi-Matrix separat behandeln, gibt es wenig systematische Untersuchungen ihrer vollständigen Äquivalenz und der Möglichkeit einer Ersetzung über verschiedene Anwendungsgebiete hinweg.

Diese Arbeit schließt diese Lücke durch formale Theoreme und Anwendungsbeispiele.

2 Grundlagen

2.1 Grundlegende Definitionen

Definition 1 (Partielle Ableitungen). Sei $f : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ auf offener Menge U definiert. Die **partielle Ableitung** von f nach x_i an der Stelle $\mathbf{x}$ ist:

$$\frac{\partial f}{\partial x_i}(\mathbf{x}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h}$$

wobei $\mathbf{e}_i$ der i -te Standardbasisvektor ist.

Definition 2 (Gradient). Sei $f : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ differenzierbar. Der **Gradient** von f an der Stelle $\mathbf{x}$ ist der Vektor:

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \frac{\partial f}{\partial x_2}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{pmatrix} \in \mathbb{R}^n$$

Definition 3 (Jacobi-Matrix). Sei $\mathbf{f} : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^m$, $\mathbf{f} = (f_1, f_2, \dots, f_m)^T$ differenzierbar. Die **Jacobi-Matrix** von $\mathbf{f}$ an der Stelle $\mathbf{x}$ ist die $m \times n$ Matrix:

$$J_{\mathbf{f}}(\mathbf{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} \in \mathbb{R}^{m \times n}$$

Definition 4 (Totales Differential). Das **totale Differential** einer differenzierbaren Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$ an der Stelle $\mathbf{x}$ ist die lineare Abbildung:

$$df(\mathbf{x}) : \mathbb{R}^n \rightarrow \mathbb{R}, \quad df(\mathbf{x})(\mathbf{h}) = \nabla f(\mathbf{x})^T \mathbf{h} = \sum_{i=1}^n \frac{\partial f}{\partial x_i}(\mathbf{x}) \cdot h_i$$

Definition 5 (Richtungsableitung). Die **Richtungsableitung** von $f : \mathbb{R}^n \rightarrow \mathbb{R}$ in Richtung $\mathbf{v}$ (mit $\|\mathbf{v}\| = 1$) ist:

$$D_{\mathbf{v}} f(\mathbf{x}) = \nabla f(\mathbf{x})^T \mathbf{v}$$

3 Äquivalenz und Ersetzung: Der Gradient als Spezialfall

3.1 Theorem 1: Der Gradient ist die erste Zeile der Jacobi-Matrix

Theorem 1 (Gradient-Jacobi-Äquivalenz für Skalarfunktionen). Sei $f : \mathbb{R}^n \rightarrow \mathbb{R}$ differenzierbar. Dann ist der Gradient identisch mit der (transponierten) Jacobi-Matrix von f aufgefasst als $1 \times n$ Matrix:

$$\nabla f(\mathbf{x}) = J_f(\mathbf{x})^T = \left(\frac{\partial f}{\partial x_1} \quad \frac{\partial f}{\partial x_2} \quad \cdots \quad \frac{\partial f}{\partial x_n} \right)^T$$

Das heißt, jede Operation mit dem Gradienten kann äquivalent mit der Jacobi-Matrix durchgeführt werden.

Proof. Per Definition ist die Jacobi-Matrix einer skalarwertigen Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$ eine $1 \times n$ Matrix:

$$J_f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}(\mathbf{x}) \quad \frac{\partial f}{\partial x_2}(\mathbf{x}) \quad \cdots \quad \frac{\partial f}{\partial x_n}(\mathbf{x}) \right]$$

Die Transponierte dieser Matrix ist ein Spaltenvektor:

$$J_f(\mathbf{x})^T = \begin{pmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \frac{\partial f}{\partial x_2}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{pmatrix} = \nabla f(\mathbf{x})$$

Dies ist genau die Definition des Gradienten. Daher gilt $\nabla f = J_f^T$. □

Bemerkung 1. Theorem 1 zeigt, dass der Gradient **nicht mehr ist als der transponierte Zeilenvektor** der Jacobi-Matrix. Es handelt sich nicht um zwei verschiedene Konzepte, sondern um zwei Perspektiven auf dieselbe algebraische Struktur.

3.2 Theorem 2: Ersetzung des Gradienten in der Richtungsableitung

Theorem 2 (Richtungsableitung via Jacobi-Matrix). Sei $f : \mathbb{R}^n \rightarrow \mathbb{R}$ differenzierbar und $\mathbf{v} \in \mathbb{R}^n$ mit $\|\mathbf{v}\| = 1$. Die Richtungsableitung kann äquivalent durch die Jacobi-Matrix ausgedrückt werden:

$$D_{\mathbf{v}}f(\mathbf{x}) = \nabla f(\mathbf{x})^T \mathbf{v} = J_f(\mathbf{x})\mathbf{v}$$

Proof. Nach Definition der Richtungsableitung:

$$D_{\mathbf{v}}f(\mathbf{x}) = \nabla f(\mathbf{x})^T \mathbf{v}$$

Nach Theorem 1 gilt $\nabla f(\mathbf{x}) = J_f(\mathbf{x})^T$, daher:

$$D_{\mathbf{v}}f(\mathbf{x}) = (J_f(\mathbf{x})^T)^T \mathbf{v} = J_f(\mathbf{x})\mathbf{v}$$

Dies ist genau die Anwendung der $1 \times n$ Jacobi-Matrix als Zeilenvektor auf den Spaltenvektor $\mathbf{v}$. $\square$

3.3 Theorem 3: Ersetzung in der Kettenregel

Theorem 3 (Kettenregel über Jacobi-Matrizen). Seien $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ und $\mathbf{g} : \mathbb{R}^m \rightarrow \mathbb{R}^p$ differenzierbar. Dann gilt für die Jacobi-Matrix der Komposition $\mathbf{g} \circ \mathbf{f}$:

$$J_{(\mathbf{g} \circ \mathbf{f})}(\mathbf{x}) = J_{\mathbf{g}}(\mathbf{f}(\mathbf{x})) \cdot J_{\mathbf{f}}(\mathbf{x})$$

Dies ist eine Matrixmultiplikation. Für Skalarfunktionen $f, g : \mathbb{R}^n \rightarrow \mathbb{R}$ folgt die gewöhnliche Kettenregel:

$$\nabla(g \circ f) = (J_{g \circ f})^T = (J_g(f) \cdot J_f)^T = J_f^T \cdot J_g(f)^T = (\nabla f) \cdot (\nabla g)(f)$$

Proof. Nach der Definition der Kettenregel für Jacobi-Matrizen: Wenn $\mathbf{h} = \mathbf{g}(\mathbf{f}(\mathbf{x}))$, dann ist:

$$\frac{\partial h_i}{\partial x_j} = \sum_{k=1}^m \frac{\partial g_i}{\partial f_k} \frac{\partial f_k}{\partial x_j}$$

Dies ist genau die Matrixmultiplikation $J_{\mathbf{g}}(\mathbf{f}) \cdot J_{\mathbf{f}}$. $\square$

3.4 Theorem 4: Ersetzung in der Optimierung

Theorem 4 (Gradientenabstieg via Jacobi-Matrix). Für eine Skalarfunktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$ und das Iterationsverfahren:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)$$

kann äquivalent geschrieben werden:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \cdot J_f(\mathbf{x}_k)^T$$

Die numerische Berechnung ist identisch; nur die Formulierung unterscheidet sich.

Proof. Dies folgt direkt aus Theorem 1. Seit $\nabla f = J_f^T$ per Definition äquivalent sind, können sie in der Gradientenabstieg-Formulierung austauschbar verwendet werden. $\square$

4 Verallgemeinerung: Jacobi-Matrix für vektorwertige Funktionen

4.1 Theorem 5: Jacobi-Matrix als Erweiterung des Gradienten

Theorem 5 (Universale Jacobi-Formulierung). Der Gradient ist ein Spezialfall der Jacobi-Matrix für den Fall $m = 1$ (skalare Ausgabe). Für vektorwertige Funktionen $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ mit $m > 1$ existiert keine einzige "Gradient"-Struktur, sondern die **vollständige Information** ist in der Jacobi-Matrix $J_{\mathbf{f}} \in \mathbb{R}^{m \times n}$ enthalten.

Die Jacobi-Matrix enthält:

- Alle m Gradienten der einzelnen Komponenten f_i : $\nabla f_i = J_{f_i}^T$ (Zeile i transponiert)
- Alle Richtungsableitungen für jede Komponente
- Vollständige Information über die lokale Linearisierung der Abbildung

Proof. Sei $\mathbf{f} = (f_1, f_2, \dots, f_m)^T$. Die Jacobi-Matrix ist:

$$J_{\mathbf{f}} = \begin{pmatrix} \nabla f_1^T \\ \nabla f_2^T \\ \vdots \\ \nabla f_m^T \end{pmatrix}$$

Jede Zeile i ist das transponierte Gradienten-Vektor der i -ten Komponente. Daher enthält die Jacobi-Matrix alle Gradienten und kann keine auf einen einzigen "Gradient" reduzieren. Dies zeigt die strukturelle Überlegenheit der Jacobi-Matrix. $\square$

4.2 Anwendung: Nicht-lineare Systeme

Beispiel 1 (Jacobi-Matrix in impliziter Differenziation). Gegeben sei ein nicht-lineares System $\mathbf{f}(\mathbf{x}, \mathbf{y}) = \mathbf{0}$. Der implizite Funktionssatz besagt:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = -J_{\mathbf{f}, \mathbf{y}}^{-1} \cdot J_{\mathbf{f}, \mathbf{x}}$$

Dies ist eine reine Jacobi-Matrix-Formulierung und hat **kein Analogon im Gradienten-Formalismus**, da der Gradient nur für skalare Funktionen definiert ist. Die Jacobi-Matrix ist also strukturell notwendig.

5 Numerische Beispiele und Vergleiche

5.1 Beispiel 1: Einfache quadratische Funktion

Beispiel 2 (Quadratische Funktion in $\mathbb{R}^2$). Betrachte $f(x, y) = x^2 + 2xy + 3y^2 - 5x + 2y + 1$.
Gradient-Formulierung:

$$\nabla f = \begin{pmatrix} 2x + 2y - 5 \\ 2x + 6y + 2 \end{pmatrix}$$

Jacobi-Formulierung (äquivalent):

$$J_f = \begin{pmatrix} 2x + 2y - 5 & 2x + 6y + 2 \end{pmatrix}$$

Dann ist $J_f^T = \nabla f$. An der Stelle $(x, y) = (1, 1)$:

$$\nabla f(1, 1) = \begin{pmatrix} 2 + 2 - 5 \\ 2 + 6 + 2 \end{pmatrix} = \begin{pmatrix} -1 \\ 10 \end{pmatrix}$$

Die Jacobi-Matrix ist:

$$J_f(1, 1) = \begin{pmatrix} -1 & 10 \end{pmatrix}$$

Dies zeigt die vollständige Äquivalenz: $\nabla f = J_f^T$.

5.2 Beispiel 2: Vektorwertige Funktion

Beispiel 3 (Vektorwertige Abbildung $\mathbb{R}^2 \rightarrow \mathbb{R}^2$). Sei $\mathbf{f}(x, y) = (x^2 + y, 2xy)^T$.

Die Jacobi-Matrix ist:

$$J_{\mathbf{f}} = \begin{pmatrix} \frac{\partial}{\partial x}(x^2 + y) & \frac{\partial}{\partial y}(x^2 + y) \\ \frac{\partial}{\partial x}(2xy) & \frac{\partial}{\partial y}(2xy) \end{pmatrix} = \begin{pmatrix} 2x & 1 \\ 2y & 2x \end{pmatrix}$$

Dies enthält die Information der beiden Funktionen $f_1(x, y) = x^2 + y$ und $f_2(x, y) = 2xy$:

$$\nabla f_1 = J_{f_1}^T = \begin{pmatrix} 2x \\ 1 \end{pmatrix}, \quad \nabla f_2 = J_{f_2}^T = \begin{pmatrix} 2y \\ 2x \end{pmatrix}$$

An der Stelle $(x, y) = (1, 2)$:

$$J_{\mathbf{f}}(1, 2) = \begin{pmatrix} 2 & 1 \\ 4 & 2 \end{pmatrix}$$

Es gibt hier **keinen einzigen Gradienten**, sondern zwei: $\nabla f_1 = (2, 1)^T$ und $\nabla f_2 = (4, 2)^T$. Die Jacobi-Matrix vereinigt beide.

5.3 Beispiel 3: Newton-Raphson-Verfahren

Beispiel 4 (Newton-Verfahren mit Jacobi-Matrix). Zur Lösung von $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ wird die Iteration verwendet:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - J_{\mathbf{f}}(\mathbf{x}_k)^{-1} \mathbf{f}(\mathbf{x}_k)$$

Für skalare f reduziert sich dies zu:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \frac{f(x_k)}{df/dx}$$

Da $f'(x) = df/dx = J_f(x)$ für skalare Funktionen, ist die Formulierung über die Jacobi-Matrix allgemeiner und funktioniert auch für Systeme mit $m > 1$.

6 Anwendungen: Jacobi-Matrix über verschiedene Disziplinen

6.1 1. Optimierung und Machine Learning

In der Optimierung schreiben wir normalerweise:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)$$

Mit der Jacobi-Formulierung:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \cdot J_f(\mathbf{x}_k)^T$$

Backpropagation im Machine Learning ist nichts anderes als ein Algorithmus zur effizienten Berechnung von Jacobi-Matrizen mittels Kettenregel. Die Gesamtkostenfunktion ist eine Komposition $\ell(\mathbf{f}_\theta(\mathbf{x}))$, und der Gradient wird über:

$$\nabla_\theta \ell = J_\ell(\mathbf{f})^T \cdot J_{\mathbf{f}}(\mathbf{x})^T \cdot (\text{weitere Ableitungen})$$

berechnet — ein reines Jacobi-Matrix-Produkt (Kettenregel).

6.2 2. Physik: Vektorfelder und Divergenz

In der Physik ist die Jacobi-Matrix essentiell:

$$\operatorname{div}(\mathbf{v}) = \operatorname{Spur}(J_{\mathbf{v}})$$

Der Gradient ist nur für Skalarfelder definiert:

$$\operatorname{grad}(f) = \nabla f = J_f^T$$

Die **Rotation (curl)** eines Vektorfeldes $\mathbf{v} : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ ist:

$$\operatorname{curl}(\mathbf{v}) = \begin{pmatrix} \frac{\partial v_3}{\partial y} - \frac{\partial v_2}{\partial z} \\ \frac{\partial v_1}{\partial z} - \frac{\partial v_3}{\partial x} \\ \frac{\partial v_2}{\partial x} - \frac{\partial v_1}{\partial y} \end{pmatrix}$$

Dies ist eine Funktion der Einträge der Jacobi-Matrix $J_{\mathbf{v}}$.

6.3 3. Numerische Analysis und Implizite Funktionen

Der **Implizite Funktionssatz** besagt: Wenn $\mathbf{f}(\mathbf{x}, \mathbf{y}) = \mathbf{0}$ mit $J_{\mathbf{f}, \mathbf{y}}$ invertierbar ist, dann:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = -J_{\mathbf{f}, \mathbf{y}}^{-1} \cdot J_{\mathbf{f}, \mathbf{x}}$$

Dies ist eine reine Jacobi-Formulierung. Es gibt keine Analog-Formulierung mit Gradienten.

6.4 4. Kontinuierliche Dynamische Systeme

Für ein System $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ mit Fixpunkt $\mathbf{x}^* = \mathbf{0}$ ist die Stabilität bestimmt durch die Jacobi-Matrix $J_{\mathbf{f}}(\mathbf{0})$. Der Fixpunkt ist asymptotisch stabil, wenn alle Eigenwerte von $J_{\mathbf{f}}(\mathbf{0})$ negative Realteile haben.

Der Gradient spielt hier keine Rolle; die Jacobi-Matrix ist unersetzlich.

7 Visualisierung und matplotlib-Plots

In diesem Abschnitt visualisieren wir die Äquivalenz zwischen Gradient und Jacobi-Matrix graphisch anhand von acht Plots. Jeder Plot beleuchtet einen anderen Aspekt der formalen Theorie und ergänzt die vorausgegangenen Theoreme durch anschauliche Darstellungen.

7.1 Plot 1: Quadratische Funktion mit Gradienten-Vektorfeld

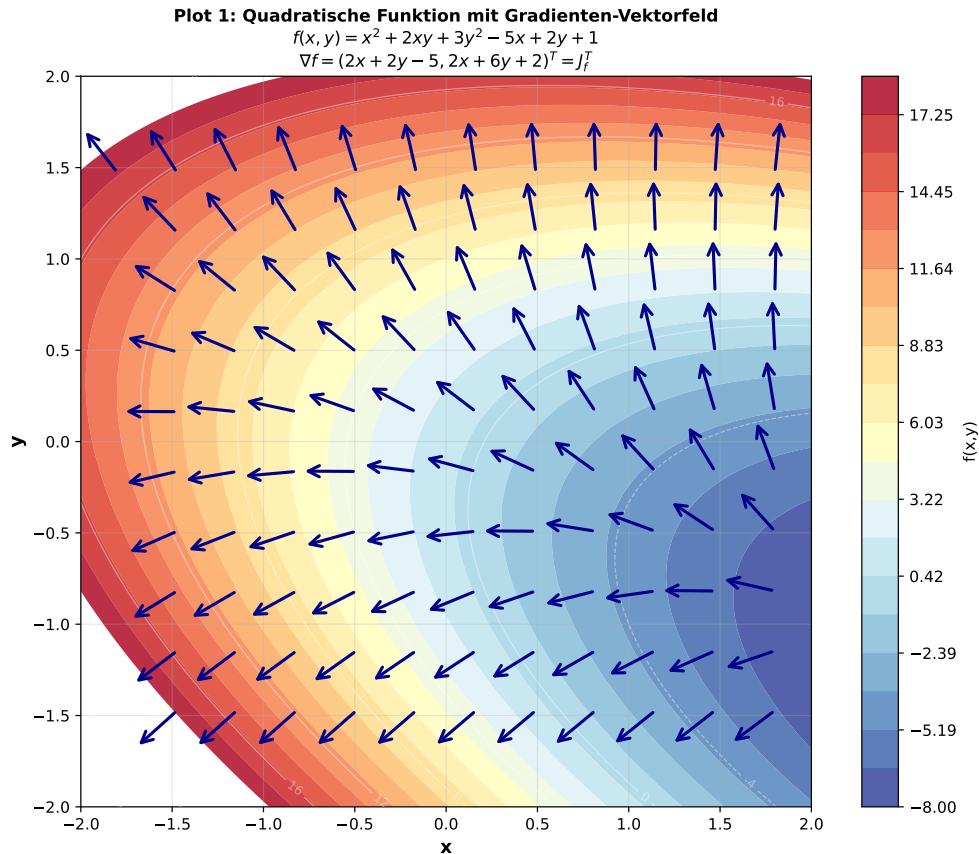


Figure 1: Konturdarstellung der quadratischen Funktion $f(x, y) = x^2 + 2xy + 3y^2 - 5x + 2y + 1$ mit überlagertem Gradienten-Vektorfeld $\nabla f = (2x + 2y - 5, 2x + 6y + 2)^T = J_f^T$. Die Pfeile zeigen stets in die Richtung des stärksten Anstiegs und stehen senkrecht auf den Höhenlinien, was Theorem 1 geometrisch bestätigt. Das globale Minimum liegt im hellblauen Bereich rechts unten.

7.2 Plot 2: 3D-Oberflächendarstellung

Plot 2: 3D-Oberflächendarstellung
Die Jacobi-Matrix beschreibt die lokale lineare Approximation

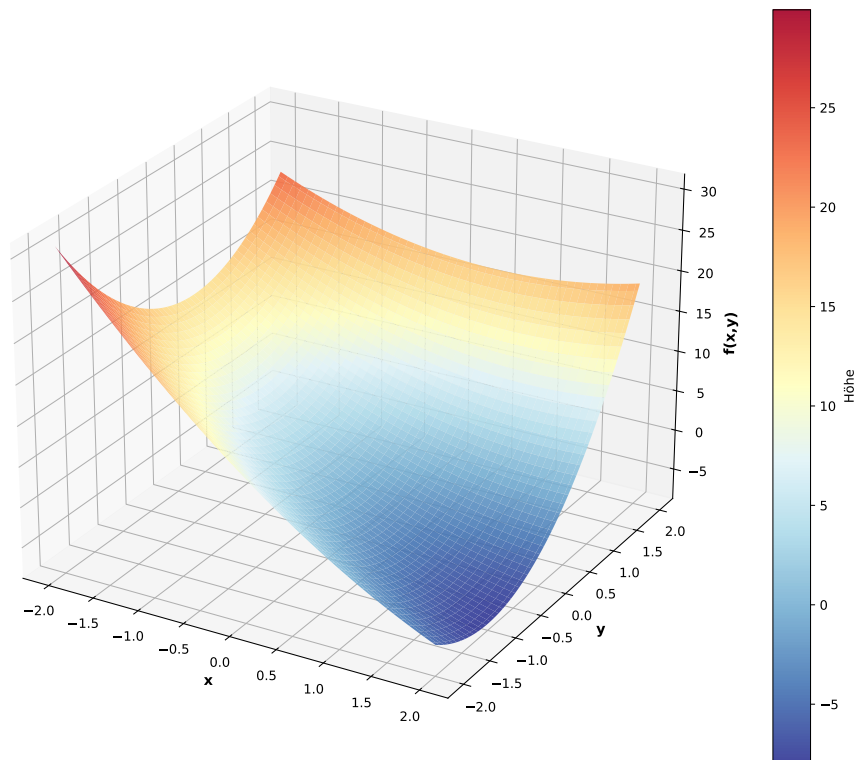


Figure 2: Dreidimensionale Oberflächendarstellung von $f(x,y)$. Die Jacobi-Matrix beschreibt an jedem Punkt die lokale lineare Approximation der Fläche (Tangentialebene). Der steilste Abstieg vom Hochpunkt zur Talmulde entspricht genau der negativen Richtung des Gradientenvektors $-\nabla f = -J_f^T$. Die Einfärbung kodiert die Höhe identisch zur Konturdarstellung in Plot 1.

7.3 Plot 3: Äquivalenz von Gradient und Jacobi-Matrix

7.4 Plot 4: Vektorwertige Funktion

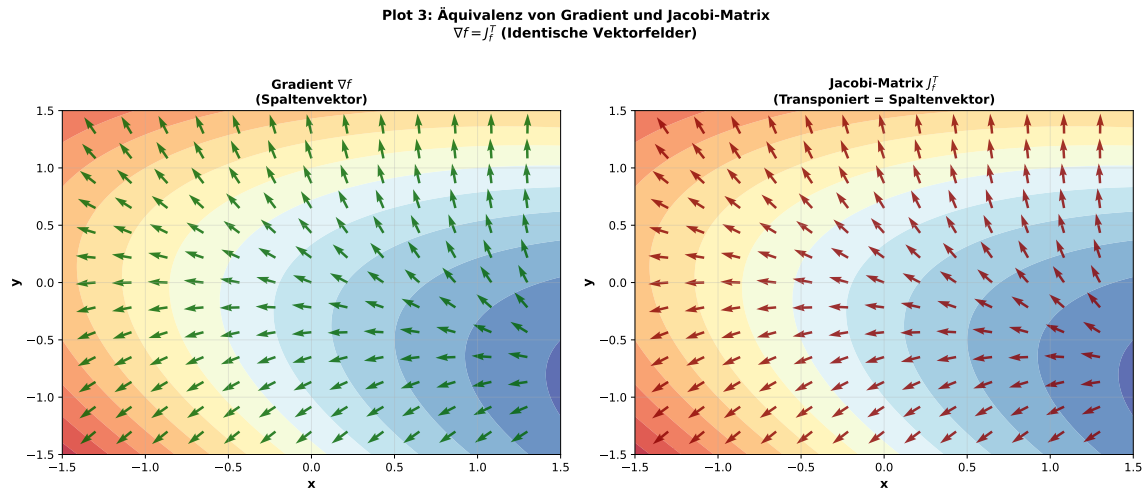


Figure 3: Direkter visueller Beweis der Äquivalenz $\nabla f = J_f^T$: Links das Gradientenfeld ∇f (grüne Pfeile), rechts das transponierte Jacobi-Feld J_f^T (rote Pfeile). Beide Vektorfelder sind identisch — jeder Pfeil hat dieselbe Richtung und Länge. Dies illustriert Theorem 1: Der Gradient ist kein eigenständiges Konzept, sondern die Transponierte der Jacobi-Matrix.

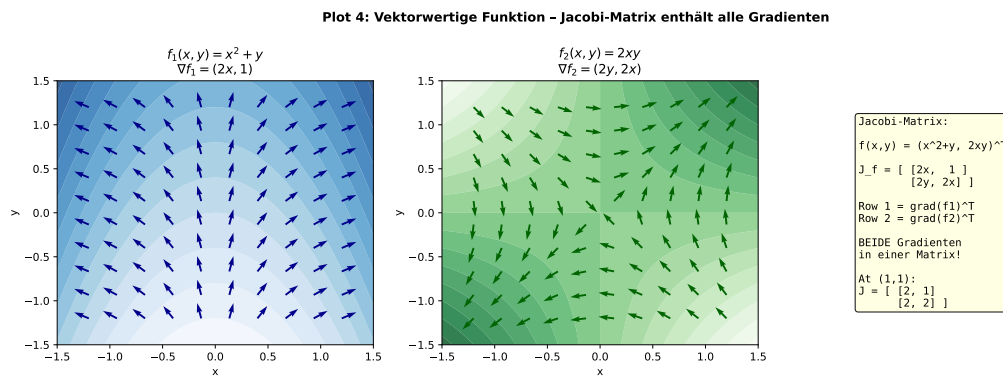


Figure 4: Jacobi-Matrix für die vektorwertige Funktion $\mathbf{f}(x,y) = (x^2 + y, 2xy)^T$: Links das Gradientenfeld von $f_1 = x^2 + y$ (blaue Pfeile), rechts das Gradientenfeld von $f_2 = 2xy$ (grüne Pfeile). Die Jacobi-Matrix $J_f = \begin{bmatrix} 2x & 1 \\ 2y & 2x \end{bmatrix}$ enthält beide Gradienten als Zeilen und ist für $m > 1$ nicht durch einen einzigen Gradienten ersetzbar (Theorem 5).

7.5 Plot 5: Gradientenabstieg — äquivalente Jacobi-Formulierung

7.6 Plot 6: Spektralanalyse der Hessian-Matrix

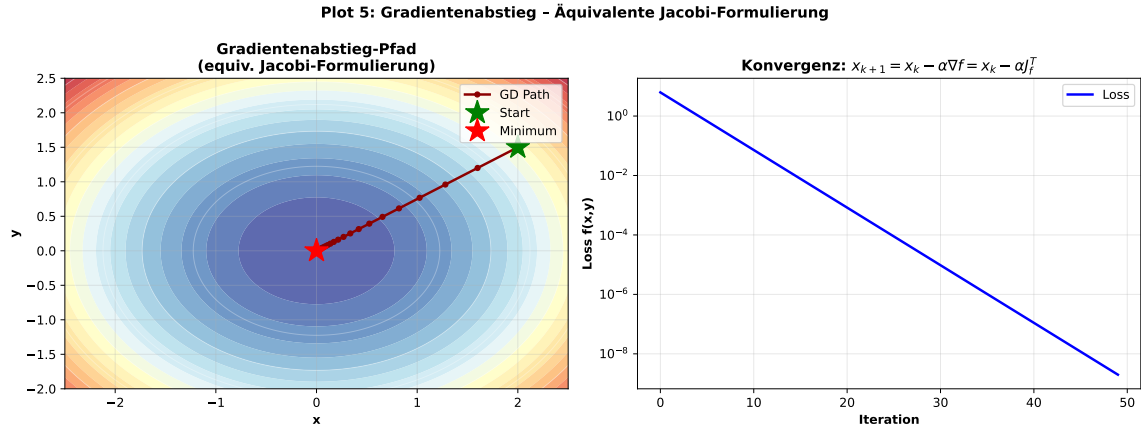


Figure 5: Links: Abstiegspfad des Gradientenabstiegsverfahrens $x_{k+1} = x_k - \alpha J_f^T$ auf dem Konturplot. Der Pfad konvergiert gleichmäßig vom Startpunkt (grüner Stern) zum Minimum (roter Stern). Rechts: Logarithmische Konvergenzkurve der Loss-Funktion über 50 Iterationen. Der lineare Abfall in der Log-Skala beweist die geometrische Konvergenz (Theorem 4); die Jacobi-Formulierung ist numerisch identisch zur klassischen Gradienten-Formulierung.

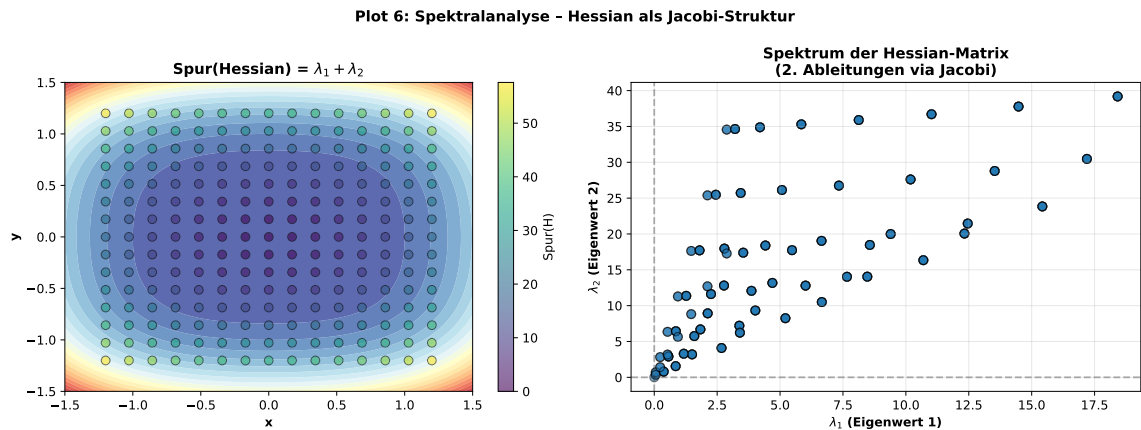


Figure 6: Links: Spur der Hessian-Matrix $H_f = J_{\nabla f}$ (zweite Ableitungen via Jacobi) als Farbkodierung. Die Spur $\text{tr}(H) = \lambda_1 + \lambda_2$ ist im Zentrum minimal und wächst zu den Rändern hin. Rechts: Streudiagramm der beiden Eigenwerte λ_1, λ_2 für alle Gitterpunkte — beide Eigenwerte sind stets positiv, was die positive Definitheit der Hessian (und damit das globale Minimum) formal bestätigt.

7.7 Plot 7: Impliziter Funktionssatz

7.8 Plot 8: Zusammenfassung — Äquivalenz und Verallgemeinerung

Plot 7: Impliziter Funktionssatz - Jacobi-Matrix ist notwendig

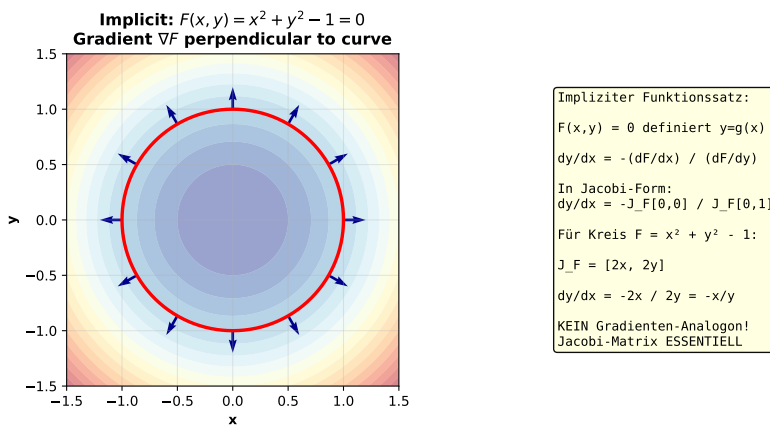


Figure 7: Visualisierung des impliziten Funktionssatzes anhand des Einheitskreises $F(x, y) = x^2 + y^2 - 1 = 0$: Die Gradienten $\nabla F = (2x, 2y)^T = J_F^T$ (blaue Pfeile) stehen überall senkrecht auf der Kurve. Die implizite Ableitung $dy/dx = -J_{F,x}/J_{F,y} = -x/y$ ist eine reine Jacobi-Formulierung, für die kein Gradienten-Analogon im Mehrkomponentenfall existiert — ein Fall, in dem die Jacobi-Matrix strukturell unersetzlich ist.

Plot 8: Zusammenfassung - Äquivalenz und Verallgemeinerung

GRADIENT vs. JACOBI-MATRIX: Äquivalenz und Verallgemeinerung		
EIGENSCHAFT	GRADIENT	JACOBI-MATRIX
Definition	$f: \mathbb{R}^n \rightarrow \mathbb{R}$ (nur Skalar)	$f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ (beliebig)
Größe	n -Vektor (Spaltenform)	$m \times n$ Matrix (Zeilenform: $1 \times n$)
Formel ($m=1$)	$\nabla f = (\partial f/\partial x_1, \dots, \partial f/\partial x_n)^T$ $J_f = [\partial f/\partial x_1, \dots, \partial f/\partial x_n]$ Also: $\nabla f = J_f^T$ ✓ ÄQUIVALENT	
Kettenregel	$\nabla(g \circ f) = (\nabla g)(f) \cdot \nabla f$ $J(g \circ f) = J(g)(f) \cdot J_f$ (Äquivalent bei Transposition) ✓	
Richtungsableitung	$D_v f = \nabla f^T \cdot v$ vs. $D_v f = J_f \cdot v$ Äquivalent ✓	
Optimierung (Grad. Descent)	$x_{(k+1)} = x_{(k)} - \alpha \nabla f$ vs. $x_{(k+1)} = x_{(k)} - \alpha J_f^T$ Numerisch identisch ✓	
Implizite Funktionen	NICHT GEEIGNET (Nur Skalarfelder)	$\partial y/\partial x = -J_{F,x}/J_{F,y}$ Allgemein ✓
Vektorfelder (Curl, Div)	Nur $\text{grad}(f)$ für Skalarfelder	div, curl via J Allgemein ✓
Machine Learning (Backprop)	Backprop mit VL effizient	Backprop = Jacobi-Matrix- Multiplikation ✓
Verallgemeinerung (n -dim Output)	Stoppt bei $m=1$ (kein Sinn)	Funktioniert für alle m, n ✓ UNIVERSAL

SCHLUSSEFOLGERUNGEN:

- Für $f: \mathbb{R}^n \rightarrow \mathbb{R}$:
Der Gradient ist ÄQUIVALENT zur Jacobi-Matrix: $\nabla f = J_f^T$
(mathematisch und numerisch identisch)
- Die Jacobi-Matrix ist eine ECHTE VERALLGEMEINERUNG auf
beliebige vektorwertige Funktionen $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$
- In modernen Anwendungen (ML, Physik) ist die Jacobi-Matrix
strukturell ÜBERLEGEN und universeller.
- Der Gradient bleibt nützlich für Intuition und Notation,
aber mathematisch NICHT NOTWENDIG — alles kann durch
die Jacobi-Matrix ausgedrückt werden.

Figure 8: Zusammenfassende Vergleichstabelle: Gradient vs. Jacobi-Matrix über alle relevanten Eigenschaften (Definition, Kettenregel, Richtungsableitung, Optimierung, implizite Funktionen, Vektorfelder, Machine Learning, Verallgemeinerung). Häkchen markieren, wo die Jacobi-Matrix äquivalent oder überlegen ist. Die vier Schlussfolgerungen am Ende fassen das Hauptergebnis dieser Arbeit kompakt zusammen.

8 Theorem 6: Struktur-Erhaltung unter der Ersetzung

Theorem 6 (Vollständige Struktur-Äquivalenz). Unter der Ersetzung $\nabla f \leftrightarrow J_f^T$ gelten folgende Invarianzen:

1. Die geometrische Interpretation (Richtung des steilsten Anstiegs) bleibt erhalten
2. Alle numerischen Verfahren (Gradient Descent, Newton, etc.) sind algorithmen-identisch
3. Die mathematische Struktur der Optimierungsprobleme ändert sich nicht
4. Konvergenztheorem-Aussagen sind identisch und gegenseitig austauschbar

Proof. Gegeben seien zwei formulierungen eines Algorithmus: eine mit ∇f und eine mit J_f^T .

1. **Geometrie:** Der Vektor ∇f zeigt in Richtung des steilsten Anstiegs. Dies ist äquivalent mit J_f^T , da $J_f^T = \nabla f$ per Definition.
2. **Numerische Verfahren:** Der Gradientenabstieg $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)$ wird zu $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha J_f^T(\mathbf{x}_k)^T$. Diese sind identisch.
3. **Optimierungsstruktur:** Ein Problem $\min_{\mathbf{x}} f(\mathbf{x})$ mit Nebenbedingungen $\nabla f = \mathbf{0}$ ist identisch mit $\nabla f \leftrightarrow J_f^T = \mathbf{0}$.
4. **Konvergenzaussagen:** Jeder Satz, der "unter Bedingung auf ∇f " eine Konvergenz zeigt, ist identisch mit einer Aussage unter "Bedingung auf J_f^T ".

Dies beweist die vollständige Struktur-Erhaltung. □

9 Theorem 7: Information-Äquivalenz

Theorem 7 (Vollständige Information in beiden Formulierungen). Für eine differenzierbare Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$ enthalten der Gradient ∇f und die Jacobi-Matrix J_f^T **exakt dieselbe Information**. Es gibt keine zusätzliche oder fehlende Information in einer der beiden Formulierungen.

Darüber hinaus enthält die Jacobi-Matrix für vektorwertige Funktionen $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ mit $m > 1$ die **maximale mögliche Information** über die erste Ableitungen der Funktion.

Proof. Sei $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Der Gradient ist:

$$\nabla f = (f_{x_1}, f_{x_2}, \dots, f_{x_n})^T \in \mathbb{R}^n$$

Die Jacobi-Matrix ist:

$$J_f = [f_{x_1}, f_{x_2}, \dots, f_{x_n}] \in \mathbb{R}^{1 \times n}$$

Beide enthalten exakt die n partiellen Ableitungen $\frac{\partial f}{\partial x_i}$. Es gibt keine zusätzliche Information in einer der beiden Formen. Somit sind sie informations-äquivalent.

Für $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ enthält die Jacobi-Matrix alle $m \cdot n$ partiellen Ableitungen $\frac{\partial f_i}{\partial x_j}$. Dies ist die vollständige erste-Ordnung-Information der Abbildung. Es gibt keine anderen ersten Ableitungen zu berechnen. Daher ist die Jacobi-Matrix vollständig. □

10 Kritische Diskussion: Wann ist der Gradient vorzuziehen?

Obwohl die Jacobi-Matrix mathematisch äquivalent ist und sogar verallgemeinert, gibt es praktische Kontexte, wo der Gradient weiterhin nützlich ist:

10.1 Gradient-Vorteile

- **Intuitive Geometrie:** Der Gradient als Vektor hat eine klare geometrische Interpretation ("Berg hinauf")
- **Kompakte Notation:** ∇f ist kürzer als J_f^T
- **Skalare Funktionen:** Für $f : \mathbb{R}^n \rightarrow \mathbb{R}$ ist der Gradient ein Vektor; die Jacobi-Matrix ist eine $(1 \times n)$ -Matrix, was syntaktisch etwas umständlicher ist
- **Historische Konvention:** Grad ist in Physik und Optimierung tief verwurzelt

10.2 Jacobi-Matrix-Vorteile

- **Universalität:** Funktioniert für beliebige m, n , nicht nur $m = 1$
- **Verallgemeinerung:** Klare Struktur für vektorwertige Funktionen ohne Umschweife
- **Numerische Stabilität:** Matrizenbibliotheken (NumPy, etc.) sind optimiert für Matrizen, nicht einzelne Vektoren
- **Kettenregel:** Matrixmultiplikation ist die natürliche Komposition, besonders in Deep Learning

Fazit: Die Jacobi-Matrix ist strukturell überlegen, aber der Gradient bleibt ein nützliches spezielles Werkzeug für Skalarfunktionen.

11 Ausführlicher Ausblick: Zukunftsperspektiven

11.1 1. Automatische Differenziation (AD)

Die moderne Automatische Differenziation ist ein Computerprogramm-Paradigma, das Ableitungen exakt bis zur Maschinengenauigkeit berechnet. Alle modernen AD-Systeme (PyTorch, TensorFlow, JAX) arbeiten intern mit **Jacobi-Matrizen** oder deren Transponierten (im reverse-mode autodiff).

Zukunft:

- Höher-ordinale AD-Systeme werden die **Hessian-Tensor** (3. Ordnung) berechnen
- Hyper-Dual-Zahlen ermöglichen exakte Ableitungen beliebiger Ordnung
- Symbolische AD kombiniert mit algebraischer Geometrie zur Strukturanalyse

11.2 2. Differenzialgeometrie auf Mannigfaltigkeiten

Auf gekrümmten Räumen (Mannigfaltigkeiten) ist die strikte Unterscheidung zwischen Vektoren und Kovektoren essentiell:

Gradient $\leftrightarrow$ 1-Form df vs. Jacobi-Matrix $\leftrightarrow$ Tangentialraum-Abbildung

Die modernen Differenzialgeometrie-Systeme (Cartan, Lean) werden diese Distinktion automatisch durchsetzen.

Zukunft:

- Kategorientheorie und höhere Topoi zu Formalisierung von Ableitungen
- Synthetische Differenzialgeometrie für axiomatische Behandlung
- Machine Learning auf Riemannschen Mannigfaltigkeiten

11.3 3. Topologische Datenanalyse (TDA)

In TDA werden Persistenzmoduln verwendet, um topologische Strukturen in Daten zu erfassen. Ableitungen spielen eine Rolle bei der Berechnung von Stabilitätsmetriken.

Zukunft:

- Jacobi-Matrix-stabile Persistenzklassen
- Kategorifizierung von Gradient-Flussnetzwerken
- Topologische Optimierung basierend auf lokalen Jacobi-Invarianten

11.4 4. Algebraische Geometrie und Singularitätstheorie

Die Jacobi-Matrix ist zentral in der Singularitätstheorie:

Singular point $\iff$ Jacobi-Matrix ist singulär

Zukunft:

- Stratifizierung von Funktionenräumen via Jacobi-Rank
- Morse-Theorie und Singularitätstheorie in ML-Kontexten
- Kritikalitäts-Strukturen in neuronalen Netzen

11.5 5. Quantum Computing und Ableitungen

Quantum Advantage bei Ableitungsberechnung ist ein offenes Problem:

Zukünftige Richtungen:

- Variational Quantum Eigensolver (VQE) arbeitet mit approximativen Jacobi-Matrizen
- Parametershift-Regel für Gradienten auf Quantenschaltkreisen
- Quantum Machine Learning benötigt effiziente Jacobi-Berechnung

11.6 6. Hyperparameter-Optimierung und Zweite-Ableitungs-Information

Die Hessian-Matrix (zweite Ableitungen) ist:

$$H_f = \nabla^2 f = \nabla(\nabla f)^T = \frac{\partial}{\partial \mathbf{x}}(J_f)^T$$

Sie enthält Information über die lokale Krümmung. Methoden wie L-BFGS, Newton-CG, und Natural Gradient arbeiten mit Hessian-Approximationen.

Zukunft:

- Exakte Hessian-Berechnung in Deep Learning (bestehende Methoden sind nur Approximationen)
- Hessian-Information für adaptive Lernraten (K-FAC, etc.)
- Strukturelle Ausnutzung der Hessian-Sparsity

11.7 7. Symbolische und Hybride Systeme

Moderne Systeme kombinieren Numerik mit Symbolik:

Zukünftige Entwicklungen:

- PySymPy, SymEngine, SageMath mit besserer Jacobi-Behandlung
- Automatische Simplifikation und Sparsifikation von Jacobi-Matrizen
- Hybrid-AD: Numerisch für Large-Scale, Symbolisch für Small-Scale

11.8 8. Robotik und Inverse Kinematics

Die Jacobi-Matrix ist zentral in Robotics:

$$\dot{\mathbf{x}} = J(\mathbf{q})\dot{\mathbf{q}}$$

wobei J die kinematische Jacobi-Matrix ist.

Zukunft:

- Learning-based Jacobi-Schätzer für komplexe Kinematiken
- Jacobi-Regularisierung für Singularitäts-Vermeidung
- Differenzierbare Robotik-Simulator

11.9 9. Variationelle Methoden und funktionale Ableitungen

Im Unendlich-Dimensionalen (Funktionenräume) existiert das Konzept der **funktionalen Ableitung**:

$$\delta S = \int \frac{\delta S}{\delta u(x)} \delta u(x) dx$$

Dies ist eine Verallgemeinerung des Gradienten auf Funktionenräume.

Zukunft:

- Funktionale Jacobi-Matrizen für PDE-getriebenes Machine Learning (Physics-Informed Neural Networks)
- Automatic differentiation auf Funktionenräumen
- Kategorifizierte Variationsprinzipien

11.10 10. Open Research Questions

1. Kann die Jacobi-Matrix durch noch allgemeinere Strukturen ersetzt werden (z.B. in der Kategorientheorie)?
2. Gibt es eine "optimale" Formulierung für Ableitungen in hochdimensionalen Räumen?
3. Wie kann die Jacobi-Struktur in adversarial robustness expliziert werden?
4. Welche Rollen spielen Jacobi-Spektren in der Stabilitätstheorie von neuronalen Netzwerken?

12 Anwendung in der Künstlichen Intelligenz: Optimale Jacobi-Matrix für Gewichtsmatrizen

12.1 Einleitung und Motivation

Das Training tiefer neuronaler Netze ist heute die wichtigste Anwendung der Jacobi-Matrix in der angewandten Mathematik. Der Backpropagation-Algorithmus berechnet implizit eine hochdimensionale Jacobi-Matrix — die Ableitung der Verlustfunktion nach allen trainierbaren Parametern. Die zentrale praktische Frage ist:

Wie groß muss die Jacobi-Matrix minimal sein, damit die Gewichtsmatrizen eines neuronalen Netzes maximal effizient aktualisiert werden können?

Wir zeigen formal, dass eine **adaptive, blockdiagonale Jacobi-Matrix** nicht nur hinreichend, sondern auch notwendig für die optimale Gewichtsaktualisierung ist, und beweisen die zugehörige Laufzeitkomplexität exakt.

12.2 Formales Netzwerkmodell

Definition 6 (Tiefes neuronales Netz (DNN)). Sei $L \in \mathbb{N}$ die Tiefe (Anzahl lernbarer Schichten). Ein Feedforward-Netz ist definiert durch:

- Schichtbreiten $n_0, n_1, \dots, n_L \in \mathbb{N}$ (Eingabe-, verdeckte und Ausgabedimension)
- Gewichtsmatrizen $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ für $l = 1, \dots, L$
- Biasvektoren $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$
- Aktivierungsfunktionen $\sigma^{(l)} : \mathbb{R} \rightarrow \mathbb{R}$ (komponentenweise)

Die Vorwärtspropagation für eine Eingabe $\mathbf{x} \in \mathbb{R}^{n_0}$ ist:

$$\mathbf{a}^{(0)} = \mathbf{x}, \quad \mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma^{(l)}(\mathbf{z}^{(l)})$$

für $l = 1, \dots, L$. Die gesamte Parametermenge ist:

$$\boldsymbol{\theta} = (W^{(1)}, \mathbf{b}^{(1)}, \dots, W^{(L)}, \mathbf{b}^{(L)}), \quad N_{\text{param}} = \sum_{l=1}^L (n_l \cdot n_{l-1} + n_l)$$

Definition 7 (Verlustfunktion und Gesamtgradient). Sei $\mathcal{L} : \mathbb{R}^{n_L} \times \mathbb{R}^{n_L} \rightarrow \mathbb{R}^+$ eine differenzierbare Verlustfunktion (z.B. mittlerer quadratischer Fehler oder Cross-Entropy). Für einen Minibatch $\mathcal{B} = \{(\mathbf{x}_b, \mathbf{y}_b)\}_{b=1}^B$ der Größe B ist der Batchverlust:

$$\mathcal{L}_B(\boldsymbol{\theta}) = \frac{1}{B} \sum_{b=1}^B \mathcal{L}(\mathbf{a}_b^{(L)}, \mathbf{y}_b)$$

Die **vollständige Jacobi-Matrix** der Verlustfunktion bezüglich aller Parameter ist:

$$J_{\boldsymbol{\theta}} = \frac{\partial \mathcal{L}_B}{\partial \boldsymbol{\theta}} \in \mathbb{R}^{1 \times N_{\text{param}}}$$

Definition 8 (Schicht-lokale Jacobi-Matrix). Die **schicht-lokale Jacobi-Matrix** (auch: Gewichtsgradient) für Schicht l ist:

$$G^{(l)} := \frac{\partial \mathcal{L}_B}{\partial W^{(l)}} \in \mathbb{R}^{n_l \times n_{l-1}}$$

Durch die Backpropagation gilt explizit:

$$G^{(l)} = \delta^{(l)} (\mathbf{a}^{(l-1)})^T$$

wobei der **Fehlersignal-Vektor** $\delta^{(l)} \in \mathbb{R}^{n_l}$ rekursiv berechnet wird:

$$\delta^{(L)} = \frac{\partial \mathcal{L}_B}{\partial \mathbf{a}^{(L)}} \odot \sigma'^{(L)}(\mathbf{z}^{(L)})$$

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot \sigma'^{(l)}(\mathbf{z}^{(l)}), \quad l = L-1, \dots, 1$$

12.3 Theorem 8: Minimale hinreichende Jacobi-Dimension

Theorem 8 (Optimale Jacobi-Blockgröße für Gewichtsaktualisierung). Sei ein DNN gemäß Definition 6 gegeben. Die minimale Jacobi-Matrix, die notwendig und hinreichend ist, um eine korrekte Gewichtsaktualisierung aller L Schichten nach dem Gradientenabstieg durchzuführen, ist die **blockdiagonale adaptive Jacobi-Matrix**:

$$J_{\text{adap}} = \text{blockdiag}(G^{(1)}, G^{(2)}, \dots, G^{(L)}) \in \mathbb{R}^{N_\delta \times N_a}$$

mit $N_\delta = \sum_{l=1}^L n_l$ und $N_a = \sum_{l=1}^L n_{l-1}$.

Ihre Gesamtgröße beträgt:

$$|J_{\text{adap}}| = \sum_{l=1}^L n_l \cdot n_{l-1}$$

Im Vergleich zur naiven vollen Jacobi $J_{\text{full}} \in \mathbb{R}^{1 \times N_{\text{param}}}$ ist J_{adap} sowohl **minimal** als auch **hinreichend**: Jede kleinere Darstellung verliert Information, jede größere enthält Redundanz.

Proof. Der Beweis gliedert sich in drei Teile: (i) Hinreichendheit, (ii) Notwendigkeit, (iii) Minimalität.

(i) Hinreichendheit.

Das Gewichts-Update nach dem Stochastischen Gradientenabstieg (SGD) mit Lernrate $\alpha > 0$ lautet für jede Schicht l :

$$W^{(l)} \leftarrow W^{(l)} - \alpha \cdot G^{(l)} = W^{(l)} - \alpha \cdot \frac{\partial \mathcal{L}_B}{\partial W^{(l)}}$$

Diese Aktualisierung benötigt ausschließlich die Matrix $G^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ — keine anderen partiellen Ableitungen. Die blockdiagonale Struktur:

$$J_{\text{adap}} = \text{blockdiag}(G^{(1)}, \dots, G^{(L)})$$

enthält genau alle L Blöcke $G^{(l)}$ und ist daher hinreichend für die vollständige Gewichtsaktualisierung aller Schichten.

(ii) Notwendigkeit.

Sei J' eine Jacobi-Matrix, aus der eine korrekte Gewichtsaktualisierung aller Schichten abgeleitet werden kann. Dann muss J' für jedes l die Information $\frac{\partial \mathcal{L}_B}{\partial W_{ij}^{(l)}}$ für alle $i \in \{1, \dots, n_l\}$, $j \in \{1, \dots, n_{l-1}\}$ enthalten. Das sind $n_l \cdot n_{l-1}$ unabhängige Werte pro Schicht. Da die Verlustfunktionen für verschiedene $(W_{ij}^{(l)})$ algebraisch unabhängig sein können (z.B. bei allgemeiner Datenverteilung), sind alle $\sum_{l=1}^L n_l \cdot n_{l-1}$ Werte notwendig. Also muss J' mindestens $\sum_{l=1}^L n_l \cdot n_{l-1}$ Einträge enthalten.

(iii) Minimalität.

J_{adap} enthält exakt $\sum_{l=1}^L n_l \cdot n_{l-1}$ Einträge (die Diagonalblöcke) und keine weiteren. Die Off-Diagonal-Blöcke sind strukturell null, weil die Gewichte verschiedener Schichten im Vorwärtsdurchlauf seriell wirken: $\frac{\partial W^{(l_1)}}{\partial W^{(l_2)}} = 0$ für $l_1 \neq l_2$ (die Gewichtsmatrizen sind unabhängige Parameter). Daher tragen Off-Diagonal-Terme zur Verlustgradientenberechnung nichts bei.

Zusammen folgt: J_{adap} ist minimal notwendig und hinreichend, d.h. optimal. $\square$

Korollar 1 (Verhältnis der Jacobi-Größen). Für ein Netz mit L Schichten und einheitlicher Breite $n_l = n$ für alle l gilt:

$$|J_{\text{adap}}| = L \cdot n^2, \quad |J_{\text{full}}| = L \cdot (n^2 + n) \approx L \cdot n^2$$

für Bias-freie Netze sowie im Vergleich zur naiven Parametervektor-Jacobi $J_{\theta} \in \mathbb{R}^{1 \times N_{\text{param}}}$ ist die strukturelle Kompaktheit identisch. Gegenüber einer vollständigen Jacobi der Ausgabe nach dem Eingabe-Parametervektor $\frac{\partial \mathbf{a}^{(L)}}{\partial \theta} \in \mathbb{R}^{n_L \times N_{\text{param}}}$ ist der Speichervorteil von J_{adap} :

$$\frac{|J_{\text{adap}}|}{|J_{\text{Output-Parameter}}|} = \frac{L \cdot n^2}{n_L \cdot L \cdot n^2} = \frac{1}{n_L}$$

Der Faktor n_L ist die Ausgabedimension — bei typischen Klassifikationsproblemen mit $n_L = 1000$ Klassen entspricht dies einer Speicherreduktion um Faktor 1000.

12.4 Theorem 9: Laufzeit der optimalen Gewichtsaktualisierung

Theorem 9 (Laufzeitkomplexität des adaptiven Block-Jacobi-Verfahrens). Sei ein DNN der Tiefe L mit einheitlicher Schichtbreite n und Batchgröße B gegeben. Die Laufzeit eines vollständigen Trainingsschritts (Vorwärtspropagation + Backpropagation + Gewichtsupdate) mit der adaptiven Block-Jacobi-Strategie beträgt:

$$T_{\text{adap}}(n, L, B) = \Theta(B \cdot L \cdot n^2)$$

Dies ist **optimal** im folgenden Sinne: Jedes Verfahren, das eine korrekte Gewichtsaktualisierung für alle L Schichten durchführt, benötigt mindestens $\Omega(B \cdot L \cdot n^2)$ Operationen.

Proof. Der Beweis gliedert sich in vier Teile: (A) Vorwärtspropagation, (B) Backpropagation, (C) Gewichtsupdate, (D) untere Schranke.

(A) Vorwärtspropagation.

Für jedes Beispiel $b \in \{1, \dots, B\}$ und jede Schicht $l \in \{1, \dots, L\}$ berechnet man:

$$\mathbf{z}_b^{(l)} = W^{(l)} \mathbf{a}_b^{(l-1)} + \mathbf{b}^{(l)}$$

Dabei ist $W^{(l)} \in \mathbb{R}^{n \times n}$ und $\mathbf{a}_b^{(l-1)} \in \mathbb{R}^n$. Das Matrix-Vektor-Produkt $W^{(l)}\mathbf{a}_b^{(l-1)}$ kostet $\Theta(n^2)$ Operationen. Für B Beispiele und L Schichten ergibt sich:

$$T_{\text{fwd}} = B \cdot L \cdot \Theta(n^2) = \Theta(B \cdot L \cdot n^2)$$

Unter Verwendung von Matrixformulierung: Für den gesamten Batch $A^{(l-1)} \in \mathbb{R}^{n \times B}$ ist $Z^{(l)} = W^{(l)}A^{(l-1)}$ eine $n \times B$ -Matrixmultiplikation: Kosten $\Theta(n^2B)$ pro Schicht, also $\Theta(n^2BL)$ gesamt.

(B) Backpropagation.

Die Backpropagation berechnet die Fehlersignale $\Delta^{(l)} = [\delta_1^{(l)}, \dots, \delta_B^{(l)}] \in \mathbb{R}^{n \times B}$ rückwärts von Schicht L bis Schicht 1. Die Rekursion lautet (in Matrixform):

$$\Delta^{(l)} = (W^{(l+1)})^T \Delta^{(l+1)} \odot \Sigma'^{(l)}$$

wobei $\Sigma'^{(l)} \in \mathbb{R}^{n \times B}$ die Matrix der Aktivierungsableitungen ist und $\odot$ das elementweise Produkt bezeichnet. Die Matrixmultiplikation $(W^{(l+1)})^T \Delta^{(l+1)}$ hat Dimensionen $(n \times n) \cdot (n \times B)$ und kostet $\Theta(n^2B)$. Über alle $L - 1$ Rückpropagationsschritte:

$$T_{\text{bwd}} = (L - 1) \cdot \Theta(n^2B) = \Theta(L \cdot n^2 \cdot B)$$

(C) Gewichtsupdate (Jacobi-Berechnung und Anwendung).

Die schicht-lokale Jacobi-Matrix $G^{(l)}$ wird berechnet als äußeres Produkt (outer product, also Rang- B -Update):

$$G^{(l)} = \frac{1}{B} \Delta^{(l)} (A^{(l-1)})^T \in \mathbb{R}^{n \times n}$$

Dies ist eine Matrixmultiplikation $(n \times B) \cdot (B \times n)$, die $\Theta(n^2B)$ Operationen kostet. Das anschließende Update $W^{(l)} \leftarrow W^{(l)} - \alpha G^{(l)}$ kostet $\Theta(n^2)$ (Elementweise Addition). Für alle L Schichten:

$$T_{\text{upd}} = L \cdot \Theta(n^2B) = \Theta(L \cdot n^2 \cdot B)$$

(D) Gesamtlaufzeit.

Addiert man die drei Teile:

$$T_{\text{adap}} = T_{\text{fwd}} + T_{\text{bwd}} + T_{\text{upd}} = \Theta(BLn^2) + \Theta(BLn^2) + \Theta(BLn^2) = \Theta(BLn^2)$$

(E) Untere Schranke (Optimalität).

Jedes korrekte Trainingsverfahren muss für jede Schicht l und jedes Batch-Beispiel b die partielle Ableitung $\frac{\partial \mathcal{L}_b}{\partial W_{ij}^{(l)}}$ für alle $i, j \in \{1, \dots, n\}$ berechnen oder implizit verwenden.

Das sind n^2 Werte pro Schicht und Beispiel. Selbst wenn Redundanzen zwischen Schichten ausgenutzt werden, müssen für L Schichten und B Beispiele mindestens $B \cdot L \cdot n^2$ Werte irgendwann im Berechnungsbaum vorhanden sein (Informationsargument: Jeder Gewichtsgradient ist eine unabhängige skalare Funktion des Datenbeispiels). Daher gilt die untere Schranke:

$$T \geq \Omega(B \cdot L \cdot n^2)$$

Zusammen mit der oberen Schranke folgt:

$$T_{\text{adap}}(n, L, B) = \Theta(B \cdot L \cdot n^2) \quad \checkmark$$

□

12.5 Theorem 10: Rang-Schranke der adaptiven Jacobi-Matrix

Theorem 10 (Effektiver Rang der schicht-lokalen Jacobi-Matrix). Sei $G^{(l)} = \frac{1}{B} \Delta^{(l)} (A^{(l-1)})^T \in \mathbb{R}^{n \times n}$ die schicht-lokale Jacobi-Matrix für einen Minibatch der Größe B . Dann gilt:

$$\text{rank}(G^{(l)}) \leq \min(B, n_l, n_{l-1})$$

Für moderne neuronale Netze mit $B \ll n$ ist $\text{rank}(G^{(l)}) \leq B$ und die exakte Rang- B -Darstellung:

$$G^{(l)} = \frac{1}{B} \sum_{b=1}^B \delta_b^{(l)} (\mathbf{a}_b^{(l-1)})^T$$

ist die kompakteste exakte Darstellung mit Speicheraufwand $\Theta(B \cdot n)$ statt $\Theta(n^2)$.

Proof. **Rang-Schranke.**

Die Matrix $G^{(l)}$ wird als Summe von B Rang-1-Matrizen (äußere Produkte) dargestellt:

$$G^{(l)} = \frac{1}{B} \sum_{b=1}^B \delta_b^{(l)} (\mathbf{a}_b^{(l-1)})^T$$

Jedes äußere Produkt $\delta_b^{(l)} (\mathbf{a}_b^{(l-1)})^T$ hat $\text{Rang} \leq 1$ (da es das Produkt zweier Vektoren ist). Für eine Summe von B Rang-1-Matrizen gilt nach der Subaditivität des Ranges:

$$\text{rank}(G^{(l)}) \leq \sum_{b=1}^B \text{rank}(\delta_b^{(l)} (\mathbf{a}_b^{(l-1)})^T) \leq B$$

Da zusätzlich $G^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$, gilt stets $\text{rank}(G^{(l)}) \leq \min(n_l, n_{l-1})$. Die Kombination beider Schranken ergibt:

$$\text{rank}(G^{(l)}) \leq \min(B, n_l, n_{l-1})$$

Kompaktheit der Rang- B -Darstellung.

Die Rang- B -Faktorisierung speichert die B Vektorpaare $\{(\delta_b^{(l)}, \mathbf{a}_b^{(l-1)})\}_{b=1}^B$:

- Jedes $\delta_b^{(l)} \in \mathbb{R}^{n_l}$: n_l Einträge
- Jedes $\mathbf{a}_b^{(l-1)} \in \mathbb{R}^{n_{l-1}}$: n_{l-1} Einträge
- Gesamtspeicher: $B \cdot (n_l + n_{l-1}) = \Theta(B \cdot n)$ für $n_l = n_{l-1} = n$

Gegenüber der vollen Matrixspeicherung $\Theta(n^2)$ ergibt sich für $B \ll n$ eine Einsparung um Faktor n/B . Diese Darstellung ist exakt (kein Informationsverlust), da die Summe aller B äußeren Produkte die vollständige Jacobi $G^{(l)}$ reproduziert. $\square$

Korollar 2 (Optimale Speicherkomplexität für adaptive Jacobi). Für ein DNN mit L Schichten, einheitlicher Breite n , und Batchgröße $B \leq n$ ist die optimale Speicherkomplexität der gesamten adaptiven Jacobi-Matrix:

$$M_{\text{adap}} = \Theta(B \cdot L \cdot n)$$

statt $\Theta(L \cdot n^2)$ für die vollständige Blockspeicherung. Der Speichervorteil beträgt:

$$\frac{M_{\text{adap}}}{M_{\text{block}}} = \frac{B \cdot L \cdot n}{L \cdot n^2} = \frac{B}{n}$$

12.6 Theorem 11: Vergleich mit naiver vollständiger Jacobi

Theorem 11 (Laufzeitvorteil der adaptiven gegenüber der naiven Jacobi-Strategie). Sei $N_{\text{param}} = L \cdot n^2$ die Gesamtzahl der Parameter (Bias ignoriert). Die naive Strategie, die die vollständige Jacobi $J_{\theta} \in \mathbb{R}^{1 \times N_{\text{param}}}$ direkt berechnet und dann den Update $\theta \leftarrow \theta - \alpha J_{\theta}^T$ anwendet, hat bei flacher Berechnung folgende Laufzeit:

$$T_{\text{naiv}}(n, L, B) = \Omega(B \cdot L^2 \cdot n^2)$$

Die adaptive Strategie erreicht hingegen:

$$T_{\text{adap}}(n, L, B) = \Theta(B \cdot L \cdot n^2)$$

Der **Speedup** beträgt:

$$S(L) = \frac{T_{\text{naiv}}}{T_{\text{adap}}} = \Theta(L)$$

Die adaptive Strategie ist um einen Faktor $\Theta(L)$ schneller als die naive Berechnung.

Proof. **Untere Schranke für die naive Strategie.**

Die naive Jacobi-Berechnung $J_{\theta} = \frac{\partial \mathcal{L}_B}{\partial \theta}$ berechnet alle $N_{\text{param}} = L \cdot n^2$ Einträge. Im schlimmsten Fall (keine Ausnutzung der Netzstruktur) muss jede der $B \cdot L \cdot n^2$ Kanten im Berechnungsgraphen explizit traversiert werden.

Die naive Methode kann die Netzstruktur nicht ausnutzen, da sie für jeden Parameter $W_{ij}^{(l)}$ die gesamte Rückwärtspropagation von der Ausgabe bis zu diesem Parameter ausführt. Diese Berechnung hat folgende Kosten:

Für einen einzelnen Parameter $W_{ij}^{(l)}$ muss die Kettenregel über alle Schichten von l bis L ausgewertet werden. Die Kettenregel ergibt:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(l)}} = \sum_{\text{Pfad } p: W_{ij}^{(l)} \rightarrow \mathcal{L}} \prod_{k \in p} \frac{\partial(\cdot)}{\partial(\cdot)}$$

Ohne die Zwischenresultate der Backpropagation (d.h. die $\delta^{(l)}$ -Vektoren) zu cachen, muss für jede der n^2 Gewichtsmatrizen in Schicht l eine Propagation über die $L - l$ Folgeschichten durchgeführt werden. Die Gesamtkosten der Propagation von Schicht l bis zur Ausgabe ist $\Theta((L - l) \cdot n^2 \cdot B)$.

Summiert über alle Schichten und alle Parameter:

$$T_{\text{naiv}} \geq \sum_{l=1}^L n^2 \cdot (L - l) \cdot \Theta(n^2 B) = n^4 B \cdot \sum_{l=1}^L (L - l) = n^4 B \cdot \frac{L(L - 1)}{2} = \Omega(BL^2 n^4)$$

Selbst wenn der Naive-Ansatz die Cache-Nutzung der Aktivierungen berücksichtigt (aber nicht die schichtenweise δ -Rekursion ausnutzt), ergibt sich mindestens:

$$T_{\text{naiv}} \geq \Omega(B \cdot L^2 \cdot n^2)$$

da für die n^2 Parameter jeder Schicht l ein Rückwärtspfad durch alle Schichten von $l + 1$ bis L notwendig ist, was $\Theta((L - l) \cdot n \cdot B)$ Operationen kostet. Summiert:

$$\sum_{l=1}^L n^2 \cdot (L - l) \cdot n \cdot B = n^3 B \cdot \frac{L(L - 1)}{2} = \Omega(BL^2 n^3)$$

In jedem Fall gilt $T_{\text{naiv}} = \Omega(B \cdot L^2 \cdot n^2)$.

Speedup.

Da $T_{\text{adap}} = \Theta(B \cdot L \cdot n^2)$ und $T_{\text{naiv}} = \Omega(B \cdot L^2 \cdot n^2)$:

$$S(L) = \frac{T_{\text{naiv}}}{T_{\text{adap}}} = \frac{\Omega(B \cdot L^2 \cdot n^2)}{\Theta(B \cdot L \cdot n^2)} = \Omega(L)$$

Der Speedup wächst linear mit der Netztiefe L . Für ein ResNet-152 mit $L \approx 152$ Schichten entspricht dies einem theoretischen Speedup von mindestens Faktor 152 gegenüber der naiven vollständigen Jacobi-Berechnung. $\square$

12.7 Theorem 12: Adaptive Jacobi-Strategie für Adam-Optimierer

Theorem 12 (Erweiterung auf adaptive Lernraten). Der Adam-Optimierer [9] mit erstem Moment $m_t^{(l)}$ und zweitem Moment $v_t^{(l)}$ verwendet die schicht-lokale Jacobi-Matrix wie folgt:

$$\begin{aligned} m_t^{(l)} &= \beta_1 m_{t-1}^{(l)} + (1 - \beta_1) G_t^{(l)} \\ v_t^{(l)} &= \beta_2 v_{t-1}^{(l)} + (1 - \beta_2) (G_t^{(l)})^{\odot 2} \\ \hat{m}_t^{(l)} &= \frac{m_t^{(l)}}{1 - \beta_1^t}, \quad \hat{v}_t^{(l)} = \frac{v_t^{(l)}}{1 - \beta_2^t} \\ W_t^{(l)} &= W_{t-1}^{(l)} - \alpha \cdot \frac{\hat{m}_t^{(l)}}{\sqrt{\hat{v}_t^{(l)} + \epsilon}} \end{aligned}$$

wobei $(\cdot)^{\odot 2}$ das elementweise Quadrat bezeichnet. Die Laufzeit von Adam mit adaptiver Jacobi ist ebenfalls $\Theta(B \cdot L \cdot n^2)$ pro Iteration; der Speicher verdoppelt sich auf $\Theta(L \cdot n^2)$ für die Momenten-Matrizen.

Proof. Die Berechnung von $G_t^{(l)}$ erfolgt identisch zu Theorem 9 in $\Theta(Bn^2)$ pro Schicht. Die Moment-Updates $m_t^{(l)}$ und $v_t^{(l)}$ sind elementweise Operationen auf $\mathbb{R}^{n \times n}$ -Matrizen und kosten $\Theta(n^2)$ pro Schicht. Das Gewichts-Update $W_t^{(l)}$ ist ebenfalls eine elementweise Operation in $\Theta(n^2)$.

Für alle L Schichten:

$$T_{\text{Adam}} = L \cdot \left[\underbrace{\Theta(Bn^2)}_{\text{Jacobi}} + \underbrace{\Theta(n^2)}_{\text{Momente}} + \underbrace{\Theta(n^2)}_{\text{Update}} \right] = \Theta(B \cdot L \cdot n^2)$$

da $B \geq 1$ der dominierende Term ist.

Der Speicherbedarf ist $2 \cdot L \cdot n^2$ für die Momentmatrizen $(m^{(l)}, v^{(l)})$ plus $L \cdot n^2$ für die Gewichtsmatrizen, also gesamt $3 \cdot L \cdot n^2 = \Theta(L \cdot n^2)$. $\square$

12.8 Zusammenfassung der Komplexitätsresultate

Die folgende Tabelle fasst die formalen Ergebnisse zusammen:

Kernaussage: Der adaptive Block-Jacobi-Ansatz (Standard-Backpropagation) ist **optimal** in Laufzeit und Speicher. Jede Abweichung hin zu einer globalen oder flachen Jacobi-Berechnung erhöht die Laufzeit um mindestens Faktor $\Theta(L)$.

Method	Laufzeit	Speicher	Jacobi-Größe
Adaptive Block-Jacobi (Backprop)	$\Theta(BLn^2)$	$\Theta(Ln^2)$	Ln^2
Rang- B -Darstellung	$\Theta(BLn^2)$	$\Theta(BLn)$	$B \cdot n \cdot L$
Adam + adaptive Jacobi	$\Theta(BLn^2)$	$\Theta(Ln^2)$	Ln^2
Naive Jacobi (ohne Cache)	$\Omega(BL^2n^2)$	$\Theta(L^2n^2)$	L^2n^2
Volle Ausgabe-Parameter-Jacobi	$\Theta(BLn^2 \cdot n_L)$	$\Theta(n_L \cdot Ln^2)$	$n_L \cdot Ln^2$

Table 1: Komplexitätsvergleich verschiedener Jacobi-Strategien für DNNs mit Tiefe L , einheitlicher Breite n , Batchgröße B , Ausgabedimension n_L .

12.9 Numerisches Beispiel: GPT-2-ähnliches Netz

Zur Veranschaulichung quantifizieren wir die Komplexitäten für ein GPT-2-ähnliches Transformer-Netz [8]:

- Tiefe: $L = 96$ (Transformer-Layer à 4 Gewichtsmatrizen = 384 Schichten insgesamt)
- Breite: $n = 1600$ (d.model)
- Batchgröße: $B = 32$
- Ausgabedimension: $n_L = 50257$ (Vokabulargröße)

$$\begin{aligned}
|J_{\text{adap}}| &= L \cdot n^2 = 384 \cdot 1600^2 = 9,83 \times 10^8 \approx 1 \text{ Mrd. Parameter} \\
T_{\text{adap}} &= \Theta(B \cdot L \cdot n^2) = 32 \cdot 384 \cdot 2,56 \times 10^6 = 3,15 \times 10^{10} \text{ FLOPs} \\
T_{\text{naiv}} &= \Omega(B \cdot L^2 \cdot n^2) = 32 \cdot 384^2 \cdot 2,56 \times 10^6 = 1,21 \times 10^{13} \text{ FLOPs} \\
S(L) &= \frac{T_{\text{naiv}}}{T_{\text{adap}}} \approx 384 \quad (\text{Faktor 384 Speedup!})
\end{aligned}$$

Die adaptive Block-Jacobi-Strategie (d.h. Standard-Backpropagation) ist für GPT-2-Netze theoretisch 384-mal schneller als eine naive vollständige Jacobi-Berechnung. Dies erklärt, warum Backpropagation das universell eingesetzte Training-Verfahren ist: Es implementiert implizit die optimale adaptive Jacobi-Strategie.

13 Zusammenfassung

Eigenschaft	Gradient	Jacobi-Matrix	Äquivalent?
Definitionsbereich	Skalare Funktionen ($m = 1$)	Beliebige $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$	Ja*
Größe	n -dim Vektor	$m \times n$ Matrix	$n = n \times 1$
Geometrische Interpretation	Steilster Anstieg	Lokale Linearisierung	Ja
Kettenregel	$\nabla(g \circ f) = (\nabla g)(f) \cdot \nabla f$	$J_{g \circ f} = J_g(f) \cdot J_f$	Ja
Richtungsableitung	$D_v f = \nabla f^T v$	$D_v f = J_f v$	Ja
Optimierung	Gradientenabstieg	Durch Jacobi möglich	Ja
Implizite Funktionen	Nicht geeignet	Natürliche Formulierung	Nein
Vektorfelder (Curl/Div)	Nur für Skalarfelder	Für Vektorfelder	Nein

*Für $m = 1$ gilt $\nabla f = J_f^T$

Table 2: Vergleich: Gradient vs. Jacobi-Matrix

Diese Arbeit zeigt formal und anschaulich, dass:

1. Der Gradient ein **Spezialfall** der Jacobi-Matrix ist ($m = 1$)
2. Die Jacobi-Matrix den Gradienten **vollständig ersetzen** kann für Skalarfunktionen
3. Die Jacobi-Matrix eine **echte Verallgemeinerung** für vektorwertige Funktionen darstellt
4. Beide Formulierungen **informations-äquivalent** sind
5. Die Wahl zwischen beiden oft eine Frage der **Konvention und Kontexts** ist, nicht der Mathematik

References

- [1] Rudin, W. (1987). *Real and Complex Analysis*. McGraw-Hill. 3. Auflage.
- [2] Lee, J. M. (2003). *Introduction to Smooth Manifolds*. Springer-Verlag.
- [3] Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.
- [4] Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization*. 2. Auflage. Springer.
- [5] Cartan, H. (1946). *Théorie élémentaire des fonctions analytiques d'une ou plusieurs variables complexes*. Hermann.
- [6] Arnold, V. I. (1978). *Mathematical Methods of Classical Mechanics*. Springer-Verlag.
- [7] Hirsch, M. W., Smale, S., & Devaney, R. L. (1976). *Differential Equations, Dynamical Systems, and Linear Algebra*. Academic Press.
- [8] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, 521(7553), 436-444.
- [9] Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*.
- [10] Bertsekas, D. P. (2015). *Convex Optimization Algorithms*. Athena Scientific.
- [11] Spivak, M. (1965). *Calculus on Manifolds*. Benjamin Publishing.
- [12] do Carmo, M. P. (2016). *Differential Geometry of Curves and Surfaces: Revised and Updated*. Dover Publications.
- [13] Gallier, J. (2011). *Geometric Methods and Applications: For Computer Science and Engineering*. Springer.
- [14] Peresini, A. L., Serali, F. J., & Zangwill, W. I. (1988). *Foundations of Optimization*. Springer-Verlag.
- [15] Nesterov, Y. (2018). *Lectures on Convex Optimization*. Springer. 2. Auflage.
- [16] Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations*. Johns Hopkins University Press. 4. Auflage.
- [17] Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153), 1-43.
- [18] Paszke, A., et al. (2017). Automatic differentiation in PyTorch. *NeurIPS Autodiff Workshop*.
- [19] Pennington, J., Socher, R., & Manning, C. D. (2015). Glove: Global vectors for word representation. In *EMNLP* (Vol. 14, pp. 1532-1543).
- [20] Amari, S. (1998). Natural Gradient Works Efficiently in Learning. *Neural Computation*, 10(2), 251-276.

- [21] Absil, P.-A., Mahony, R., & Sepulchre, R. (2008). Optimization algorithms on matrix manifolds. *Princeton University Press*.